파이썬 문자열

문자열은 일련의 문자입니다. 파이썬은 따옴표 안의 모든 것을 문자열로 취급합니다. 여기에는 문자, 숫자, 기호가 포함됩니다. 파이썬에는 문자 데이터 유형이 없으므로 단일 문자는 길이가 1인 문자열입니다.

```
s = "GfG"

print(s[1]) # access 2nd char
s1 = s + s[0] # update
print(s1) # print
```

산출

이 예에서 **s는** "GfG" 값을 가지고 문자열로 정의됩니다.

문자열 생성

문자열은 작은따옴표(') 나 **큰따옴표(")를** 사용하여 생성할 수 있습니다 .

```
s1 = 'GfG'
s2 = "GfG"
print(s1)
print(s2)
```

산출

```
GfG
GfG
```

다중 줄 문자열

문자열을 여러 줄에 걸쳐 표현해야 하는 경우에는 **삼중 따옴표(''' 또는 """)를** 사용할 수 있습니다 .

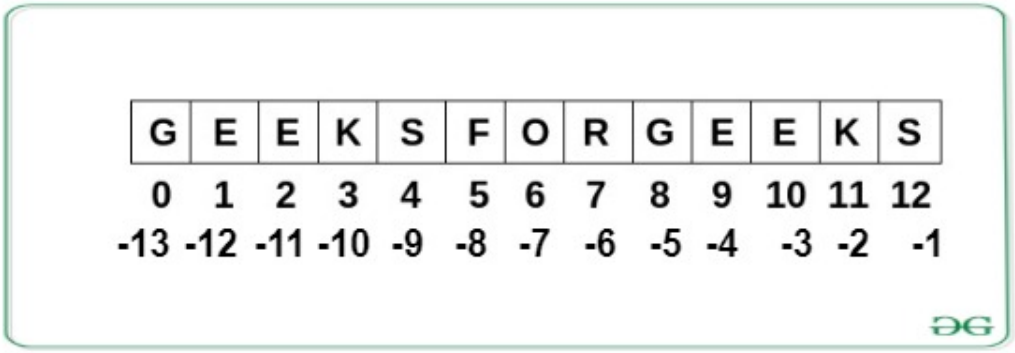
```
s = """I am Learning
Python String on GeeksforGeeks"""
print(s)

s = '''I'm a
Geek'''
print(s)
```

산출

Python 문자열에서 문자 접근하기

파이썬에서 문자열은 문자의 시퀀스이므로 **인덱싱을 사용하여 개별 문자에 접근할 수 있습니다. 문자열은** 0 부터 시작하여 end를 기준으로 -1 **까지** 인덱싱됩니다 . 이를 통해 문자열에서 특정 문자를 검색할 수 있습니다.




```
s = "GeeksforGeeks"

# Accesses first character: 'G'
print(s[0])

# Accesses 5th character: 's'
print(s[4])
```

산출

참고: 범위를 벗어난 인덱스에 접근하면 **IndexError** 가 발생합니다 . 인덱스로는 정수만 사용할 수 있으며, 부동 소수점이나 다른 유형을 사용하면 **TypeError** 가 발생합니다 .

음수 인덱싱을 사용한 문자열 액세스

Python에서는 문자열의 뒤쪽에 있는 문자에 접근하기 위해 음수 주소 참조를 허용합니다. 예를 들어 -1은 마지막 문자를 참조하고, -2는 끝에서 두 번째 문자를 참조하는 식입니다.

```
s = "GeeksforGeeks"

# Accesses 3rd character: 'k'
print(s[-10])

# Accesses 5th character from end: 'G'
print(s[-5])
```

산출

문자열 슬라이싱

슬라이싱은 **시작** 인덱스 와 **끝** 인덱스를 지정하여 문자열의 일부를 추출하는 방법입니다 . 슬라이싱 구문은 **string[start:end]** 이며 , 여기서 **start는** 시작 인덱스이고 **end는** 제외된 종료 인덱스입니다.

```
s = "GeeksforGeeks"

# Retrieves characters from index 1 to 3: 'eek'
print(s[1:4])

# Retrieves characters from beginning to index 2: 'Gee'
print(s[:3])

# Retrieves characters from index 3 to the end: 'ksforGeeks'
print(s[3:])

# Reverse a string
print(s[::-1])
```

산출

문자열 불변성

파이썬의 문자열은 불변적입니다 . 즉, 생성된 후에는 변경할 수 없습니다. 문자열을 조작해야 하는 경우 **연결, 슬라이싱, 포맷팅** 과 같은 메서드를 사용하여 원본 문자열을 기반으로 새로운 문자열을 만들 수 있습니다.

```
s = "geeksforGeeks"

# Trying to change the first character raises an error
# s[0] = 'I'  # Uncommenting this line will cause a TypeError

# Instead, create a new string
s = "G" + s[1:]
print(s)
```

산출

문자열 삭제

Python에서는 문자열이 변경 불가능하기 때문에 문자열에서 개별 문자를 삭제할 수 없습니다. 하지만 del 키워드를 사용하면 문자열 변수 전체를 삭제할 수 있습니다.

```
s = "GfG"

# Deletes entire string
del s
```

참고: del을 사용하여 문자열을 삭제한 후 **s** 에 액세스하려고 하면 변수가 더 이상 존재하지 않으므로 **NameError** 가 발생합니다 .

문자열 업데이트

문자열은 변경할 수 없으므로 문자열의 일부를 업데이트하려면 새로운 문자열을 만들어야 합니다.

```
s = "hello geeks"

# Updating by creating a new string
s1 = "H" + s[1:]

# replacnig "geeks" with "GeeksforGeeks"
```

```
s2 = s.replace("geeks", "GeeksforGeeks")
print(s1)
print(s2)
```

산출

설명:

- **s1의 경우,** 원래 문자열 **s를** 인덱스 1부터 문자열 끝까지 잘라낸 다음 "H"를 연결하여 새로운 문자열 **s1을** 생성합니다 .
- **s2의 경우,** 새로운 문자열 s2를 생성하고 replace() 메서드를 사용하여 'geeks'를 'GeeksforGeeks'로 바꿀 수 있습니다.

일반적인 문자열 메서드

Python은 문자열을 조작하는 다양한 내장 메서드를 제공합니다. 가장 유용한 메서드 몇 가지를 소개합니다.

len() : len() 함수는 문자열의 총 문자 수를 반환합니다.

```
s = "GeeksforGeeks"
print(len(s))

# output: 13
```

산출

upper() 및 lower() : upper() 메서드는 모든 문자를 대문자로 변환합니다. lower() 메서드는 모든 문자를 소문자로 변환합니다.

```
s = "Hello World"

print(s.upper())   # output: HELLO WORLD

print(s.lower())   # output: hello world
```

산출

strip() 및 replace() : strip()은 문자열의 앞뒤 공백을 제거하고 replace(old, new)는 지정된 하위 문자열의 모든 항목을 다른 문자열로 바꿉니다.

```
s = "   Gfg   "

# Removes spaces from both ends
print(s.strip())

s = "Python is fun"

# Replaces 'fun' with 'awesome'
print(s.replace("fun", "awesome"))
```

산출

문자열 연결 및 반복

+ 연산자를 사용하여 문자열을 연결할 수 있고 * 연산자를 사용하여 문자열을 반복할 수 있습니다 .

문자열은 + 연산자를 사용하여 결합할 수 있습니다 .

```
s1 = "Hello"
s2 = "World"
s3 = s1 + " " + s2
print(s3)
```

산출

*** 연산자를** 사용하면 문자열을 여러 번 반복할 수 있습니다 .

```
s = "Hello "
print(s * 3)
```

산출

문자열 서식 지정

Python은 문자열 내에 변수를 포함시키는 여러 가지 방법을 제공합니다.

f-string 사용

문자열을 포맷하는 가장 간단하고 선호되는 방법은 f-문자열을 사용하는 것입니다 .

```
name = "Alice"
age = 22
print(f"Name: {name}, Age: {age}")
```

산출

format() 사용하기

문자열을 포맷하는 또 다른 방법은 format() 메서드를 사용하는 것입니다.

```
s = "My name is {} and I am {} years old.".format("Alice", 22)
print(s)
```

산출

문자열 멤버십 테스트를 *위해* 사용

in **키워드는 문자열** 내에 특정 부분 문자열이 존재하는지 확인합니다.

```
s = "GeeksforGeeks"
print("Geeks" in s)
print("GfG" in s)
```

산출

파이썬 리스트

파이썬에서 **리스트는** 크기가 자동으로 커지거나 작아지는 내장 동적 배열입니다. 모든 유형의 항목(다른 리스트 포함)을 리스트에 저장할 수 있습니다. 리스트는 여러 유형의 항목을 포함할 수 있는데, 이는 리스트가 주로 인접한 위치에 참조를 저장하고 실제 항목은 서로 다른 위치에 저장될 수 있기 때문입니다.

- 목록에는 중복된 항목이 포함될 수 있습니다.
- 파이썬의 리스트는 변경 가능합니다. 따라서 항목을 수정, 교체 또는 삭제할 수 있습니다.
- 목록은 정렬되어 있습니다. 요소가 추가된 방식에 따라 순서가 유지됩니다.
- 목록의 항목에 접근하려면 0부터 시작하여 위치(인덱스)를 사용하여 직접 수행할 수 있습니다.

예 :

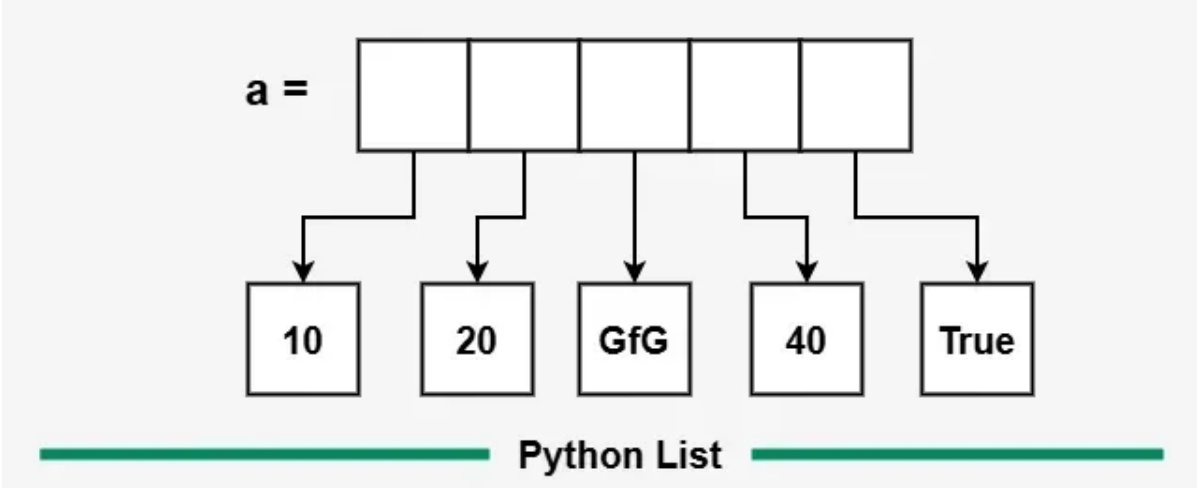
```
# Creating a Python list with different data types
a = [10, 20, "GfG", 40, True]
print(a)

# Accessing elements using indexing
print(a[0])  # 10
print(a[1])  # 20
print(a[2])  # "GfG"
print(a[3])  # 40
print(a[4])  # True

# Checking types of elements
print(type(a[2]))  # str
print(type(a[4]))  # bool
```

설명:

- 목록에는 정수(10, 20, 40), 문자열("GfG"), 부울(True)이 혼합되어 있습니다.
- 목록이 인쇄되고 개별 요소는 인덱스(0부터 시작)를 사용하여 액세스됩니다.
- type(a[2])는 "GfG"가 str임을 확인합니다.
- type(a[4])는 True가 bool임을 확인합니다.



파이썬 리스트

> ***참고: 목록은 값이 아닌 참조를 저장합니다.***
>
> *리스트의 각 요소는 리스트 구조 내부에 직접 저장되지 않습니다. 대신, 리스트는 메모리에 있는 실제 객체에 대한 참조(포인터)를 저장합니다.* ***예시*** *(이미지 표현)*
>
> - *목록 a 자체는 실제 값에 대한 참조(주소)를 담고 있는 컨테이너입니다.*
> - *Python은 내부적으로 10, 20, "GfG", 40, True에 대한 별도의 객체를 생성한 다음 해당 메모리 주소를 a에 저장합니다.*
> - *이는 요소를 수정해도 다른 요소에는 영향을 미치지 않지만 변경 가능한 경우 참조된 객체에는 영향을 미칠 수 있음을 의미합니다.*

목록 만들기

목록을 만드는 몇 가지 일반적인 방법은 다음과 같습니다.

대괄호 사용

```
# List of integers
a = [1, 2, 3, 4, 5]

# List of strings
b = ['apple', 'banana', 'cherry']

# Mixed data types
c = [1, 'hello', 3.14, True]

print(a)
print(b)
print(c)
```

산출

list() 생성자 사용

또한, 반복 가능한 객체 (문자열 , 튜플 **또는** 다른 **리스트**) 를 list() 함수 에 전달하여 리스트를 생성할 수도 있습니다 .

```
# From a tuple
a = list((1, 2, 3, 'apple', 4.5))
print(a)
```

반복되는 요소를 사용하여 목록 만들기

곱셈 연산자를 사용하여 반복되는 요소로 구성된 목록을 만들 수 있습니다.

```
# Create a list [2, 2, 2, 2, 2]
a = [2] * 5

# Create a list [0, 0, 0, 0, 0, 0, 0]
b = [0] * 7

print(a)
print(b)
```

산출

```
[2, 2, 2, 2, 2]
[0, 0, 0, 0, 0, 0, 0]
```

목록 요소 액세스

리스트의 요소에는 **인덱싱을** 사용하여 접근할 수 있습니다 . 파이썬 인덱스는 **0** 부터 시작 하므로 **a[0]은** 첫 번째 요소에 접근하고, **음수 인덱싱은** 리스트의 끝에서부터 요소에 접근할 수 있도록 합니다. 인덱스 -1은 리스트의 마지막 요소를 나타냅니다.

```
a = [10, 20, 30, 40, 50]

# Access first element
print(a[0])
```

```
# Access last element
print(a[-1])
```

산출

```
10
50
```

목록에 요소 추가

다음 방법을 사용하여 목록에 요소를 추가할 수 있습니다.

- append(): 목록의 끝에 요소를 추가합니다.
- extend(): 목록의 끝에 여러 개의 요소를 추가합니다.
- insert() : 특정 위치에 요소를 추가합니다.

```
# Initialize an empty list
a = []

# Adding 10 to end of list
a.append(10)
print("After append(10):", a)

# Inserting 5 at index 0
a.insert(0, 5)
print("After insert(0, 5):", a)

# Adding multiple elements  [15, 20, 25] at the end
a.extend([15, 20, 25])
print("After extend([15, 20, 25]):", a)
```

목록에 요소 업데이트

인덱스를 사용하여 요소에 접근하면 요소의 값을 변경할 수 있습니다.

```
a = [10, 20, 30, 40, 50]

# Change the second element
a[1] = 25
print(a)
```

산출

```
[10, 25, 30, 40, 50]
```

목록에서 요소 제거

다음을 사용하여 목록에서 요소를 제거할 수 있습니다.

- remove() : 요소의 첫 번째 발생 항목을 제거합니다.
- pop() : 특정 인덱스에 있는 요소를 제거합니다. 인덱스가 지정되지 않은 경우에는 마지막 요소를 제거합니다.
- del 문 : 지정된 인덱스의 요소를 삭제합니다.

```
a = [10, 20, 30, 40, 50]

# Removes the first occurrence of 30
a.remove(30)
print("After remove(30):", a)

# Removes the element at index 1 (20)
popped_val = a.pop(1)
print("Popped element:", popped_val)
print("After pop(1):", a)

# Deletes the first element (10)
del a[0]
print("After del a[0]:", a)
```

산출

```
remove(30) 후 : [10, 20, 40, 50]
팝된 요소: 20
pop(1) 후: [10, 40, 50]
del a[0] 이후: [40, 50]
```

목록 반복

for 루프 나 다른 반복 메서드를 사용하면 리스트를 쉽게 반복할 수 있습니다 . 리스트를 반복하는 것은 각 항목에 대해 어떤 연산을 수행하거나 특정 조건에 따라 특정 항목에 접근하려고 할 때 유용합니다. **for 루프를** 사용하여 리스트를 반복하는 예를 살펴보겠습니다 .

for 루프 사용

```
a = ['apple', 'banana', 'cherry']
```

```
a = [ apple ,  banana ,  cherry ]

# Iterating over the list
for item in a:
    print(item)
```

산출

파이썬의 중첩 리스트

중첩 리스트는 다른 리스트 안에 있는 리스트로, 행렬이나 표를 표현하는 데 유용합니다. 인덱스를 연결하여 중첩된 요소에 접근할 수 있습니다.

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

# Access element at row 2, column 3
print(matrix[1][2])
```

산출

```
6
```

파이썬의 리스트 Comprehension

리스트 컴프리헨션은 한 줄의 코드로 리스트를 생성하는 간결한 방법입니다. 리스트나 범위와 같은 반복 가능한 객체의 항목에 연산이나 필터를 적용할 때 유용합니다.

```
# Create a list of squares from 1 to 5
squares = [x**2 for x in range(1, 6)]
print(squares)
```

산출

```
[1, 4, 9, 16, 25]
```

설명:

- **for x in range(1, 6): 1에서 5까지** 각 숫자 (6 제외) 를 반복합니다 .
- **x**2:** 각 숫자 x를 제곱합니다.
- **[ ]:** 모든 제곱 숫자를 새 목록으로 수집합니다.

파이썬의 List Comprehension

리스트 컴프리헨션은 간결한 구문을 사용하여 리스트를 생성하는 방법입니다. 기존 **반복 가능 객체** (예: **리스트** 또는 **범위)의 각 항목에 표현식을** 적용하여 새 리스트를 생성할 수 있습니다 . 이를 통해 기존의 반복 기법보다 깔끔하고 읽기 쉬운 코드를 작성할 수 있습니다.

예를 들어 , 정수 목록이 있고 각 요소의 제곱을 포함하는 새 목록을 만들고 싶다면 목록 이해를 사용하면 쉽게 이를 달성할 수 있습니다.

```
a = [2,3,4,5]

res = [val ** 2 for val in a]

print(res)
```

산출

```
[4, 9, 16, 25]
```

리스트 이해의 구문



- **표현식:** 새 목록에 포함될 변환이나 값입니다.
- **item:** 반복 가능한 항목에서 가져온 현재 요소입니다.
- **반복 가능:** 시퀀스 또는 컬렉션(예: 목록, 튜플, 집합).
- **if condition (선택 사항):** 현재 항목을 포함할지 여부를 결정하는 필터링 조건입니다.

이 구문을 사용하면 반복, 수정, 조건 필터링을 모두 한 줄에 결합할 수 있습니다.

For 루프 vs. 리스트 이해

가장 큰 차이점은 for 루프는 항목을 반복하고 각 항목을 수동으로 추가하여 새 목록을 만들려면 여러 줄이 필요하다는 것입니다 . 반면, **리스트 컴퍼헨션은** 같은 작업을 한 줄로 처리하기 때문에 코드가 더 간단하고 읽기 쉽습니다.

예: 주어진 목록의 각 숫자를 두 배로 늘려 새로운 목록으로 만들고 싶은 예를 살펴보겠습니다.

for 루프를 사용합니다.

```
a = [1, 2, 3, 4, 5]

# Create an empty list 'res' to store results
res = []

# Iterate over each element in list 'a'
for val in a:

    # Multiply each element by 2 and append it to 'res'
    res.append(val * 2)

print(res)
```

산출

```
[2, 4, 6, 8, 10]
```

설명: 결과를 저장하기 위해 빈 목록 ' **res** '**를 만들고 목록** ' **a** ' 의 각 요소를 반복하고 목록 ' **a** ' 의 각 항목에 대해 2를 곱한 후 append() 메서드를 사용하여 ' **res** ' 에 추가합니다 .

리스트 이해 사용:

```
a = [1, 2, 3, 4, 5]

res = [val * 2 for val in a]

print(res)
```

산출

```
[2, 4, 6, 8, 10]
```

설명: 위의 리스트 내포에서 반복 가능한 항목은 리스트 ' **a**'**이고**, 표현식은 **val * 2로**, 리스트의 각 값을 2로 곱합니다.

리스트 컴퍼헨션 에는 특정 기준에 따라 항목을 필터링하거나 수정하는 조건문을 포함할 수 있습니다 . 이러한 조건문을 사용하면 사용자 지정 리스트를 빠르게 생성하고 코드를 더욱 깔끔하고 효율적으로 만들 수 있습니다.

예: 주어진 목록에서 모든 짝수 목록을 필터링하고 싶다고 가정해 보겠습니다.

```
a = [1, 2, 3, 4, 5]

res = [val for val in a if val % 2 == 0]

print(res)
```

산출

```
[2, 4]
```

리스트 이해의 예

범위에서 목록 만들기

간단한 예로 0부터 9까지의 숫자 목록을 만드는 것입니다.

```
# Creates a list of numbers from 0 to 9
a = [i for i in range(10)]

print(a)
```

산출

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

중첩 루프 사용

리스트 컴프리헨션은 중첩된 루프에도 사용할 수 있습니다. 여기서는 간단한 3x3 격자에 대한 좌표 쌍 리스트를 생성합니다.

```
```

```
# Creates a list of tuples representing all combinations of (x, y)
# where both x and y range from 0 to 2.
coordinates = [(x, y) for x in range(3) for y in range(3)]

print(coordinates)
```

산출

```
[(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2), (2, 0), (2, 1), (2, 2)]
```

목록의 목록 평면화

목록의 목록이 있고 이를 단일 목록으로 변환하고 싶다고 가정해 보겠습니다.

```
mat = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

res = [val for row in mat for val in row]

print(res)
```

산출

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

설명: [mat의 행에 대한 값, 행에 대한 값] 줄은 중첩된 리스트 컴프리헨션을 사용하여 **mat** 의 각 행을 반복합니다 . 각 행에 대해 해당 행의 각 **값을** 반복하면서 모든 값을 단일 리스트로 수집합니다.

파이썬 튜플

파이썬의 튜플은 변경 불가능한 순서 있는 요소의 컬렉션입니다.

- 튜플은 리스트와 비슷하지만, 리스트와는 달리 생성된 후에는 변경할 수 없습니다 (즉, 변경 불가능합니다).
- 튜플은 다양한 데이터 유형의 요소를 보관할 수 있습니다.
- The main characteristics of tuples are being **ordered** , **heterogeneous** and **immutable**.

튜플 생성

튜플은 모든 항목을 괄호() 안에 쉼표로 구분하여 만듭니다. 튜플은 여러 항목을 포함할 수 있으며, 각 항목은 서로 다른 데이터 유형을 가질 수 있습니다 .

예:

```
tup = ()
print(tup)

# Using String
tup = ('Geeks', 'For')
print(tup)

# Using List
li = [1, 2, 4, 5, 6]
print(tuple(li))

# Using Built-in Function
tup = tuple('Geeks')
print(tup)
```

혼합된 데이터 유형을 사용하여 튜플 만들기.

튜플에는 튜플, 리스트 , 사전, 심지어 함수 등 다양한 데이터 유형의 요소가 포함될 수 있습니다 .

예:

```
tup = (5, 'Welcome', 7, 'Geeks')
print(tup)

# Creating a Tuple with nested tuples
tup1 = (0, 1, 2, 3)
tup2 = ('python', 'geek')
tup3 = (tup1, tup2)
print(tup3)

# Creating a Tuple with repetition
tup1 = ('Geeks',) * 3
print(tup1)

# Creating a Tuple with the use of loop
tup = ('Geeks')
n = 5
for i in range(int(n)):
    tup = (tup,)
    print(tup)
```

튜플 접근

튜플의 요소에는 리스트의 요소에 접근하는 방식과 유사하게 인덱싱과 슬라이싱을 사용하여 접근할 수 있습니다 . 인덱싱은 첫 번째 요소의 경우 0부터 시작하여 n-1까지 올라가는데, 여기서 n은 튜플의 요소 개수입니다. 음수 인덱싱은 마지막 요소의 경우 -1부터 시작하여 역순으로 올라갑니다.

예:

```
# Accessing Tuple with Indexing
tup = tuple("Geeks")
print(tup[0])

# Accessing a range of elements using slicing
print(tup[1:4])
print(tup[:3])

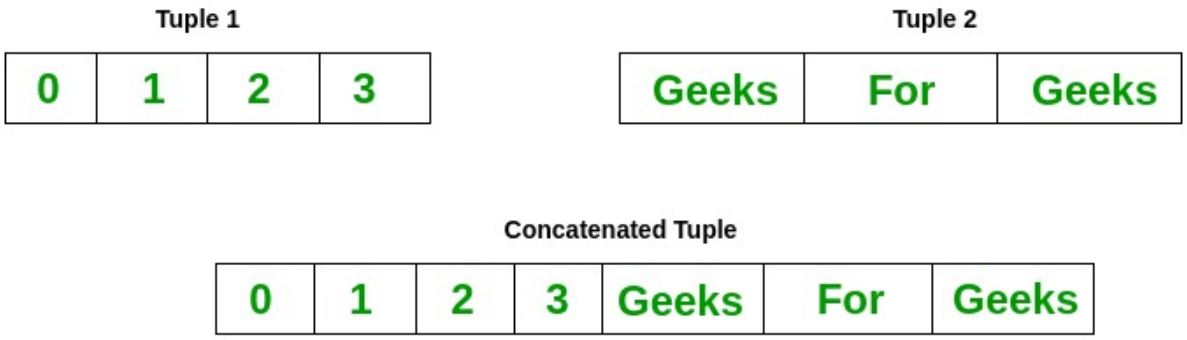
# Tuple unpacking
tup = ("Geeks", "For", "Geeks")

# This line unpack values of Tuple1
a, b, c = tup
print(a)
print(b)
print(c)
```

튜플의 연결

튜플은 + 연산자를 사용하여 연결할 수 있습니다. 이 연산은 두 개 이상의 튜플을 결합하여 새로운 튜플을 생성합니다.

> ***참고:*** *연결은 동일한 데이터 유형만 결합할 수 있으며, 리스트와 튜플을 결합하면 오류가 발생합니다.*



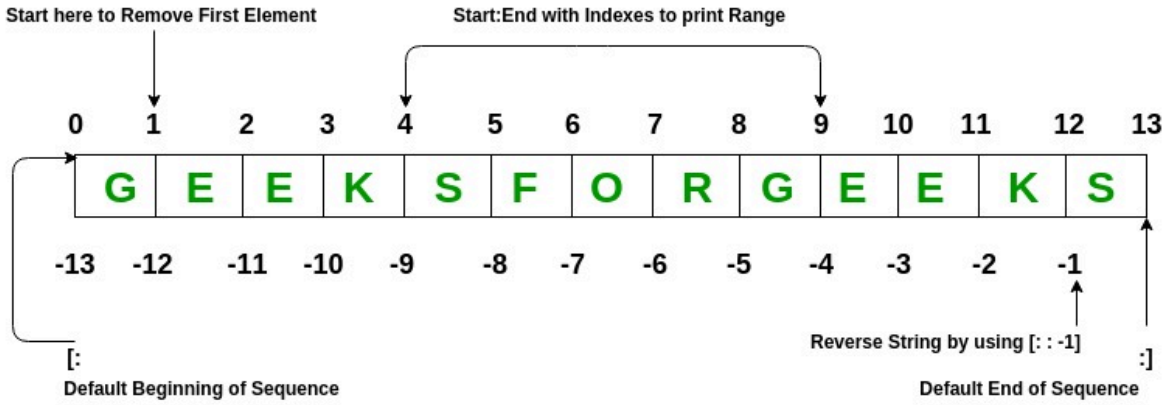

```
tup1 = (0, 1, 2, 3)
tup2 = ('Geeks', 'For', 'Geeks')

tup3 = tup1 + tup2
print(tup3)
```

튜플의 슬라이싱

튜플을 슬라이싱한다는 것은 원래 튜플의 요소 부분집합에서 새로운 튜플을 생성하는 것을 의미합니다. 슬라이싱 구문은 tuple[start:stop:step]입니다.

> ***참고*** *- 음수 증가 값은 튜플의 순서를 반대로 바꾸는 데에도 사용할 수 있습니다.*




```
tup = tuple('GEEKSFORGEEKS')

# Removing First element
print(tup[1:])

# Reversing the Tuple
print(tup[::-1])
```

```
# Printing elements of a Range
print(tup[4:9])
```

산출

```
('E', 'E', 'K', 'S', 'F', 'O', 'R', 'G', 'E', 'E', 'K', 'S')
('S', 'K', 'E', 'E', 'G', 'R', 'O', 'F', 'S', 'K', 'E', 'E', 'G')
('S', 'F', 'O', 'R', 'G')
```

튜플 삭제

튜플은 변경 불가능하므로 튜플의 개별 요소를 삭제할 수 없습니다. 그러나 del 문을 사용하면 튜플 전체를 삭제할 수 있습니다 .

> ***참고:*** *튜플을 삭제한 후 인쇄하면 오류가 발생합니다.*

```
tup = (0, 1, 2, 3, 4)
del tup

print(tup)
```

별표(*)를 사용한 튜플 압축 해제

Python에서 "*" 연산자는 튜플을 풀 때 여러 항목을 하나의 리스트로 가져오는 데 사용할 수 있습니다. 특정 요소 몇 개만 추출하고 나머지는 하나로 모으고 싶을 때 유용합니다.

```
tup = (1, 2, 3, 4, 5)

a, *b, c = tup

print(a)
print(b)
print(c)
```

산출

```
1
[2, 3, 4]
5
```

설명:

- **a가** 첫 번째 항목을 얻습니다.
- **c는** 마지막 항목을 얻습니다.
- ***b는** 그 사이의 모든 것을 목록으로 수집합니다.

파이썬의 Dictionaries

파이썬 딕셔너리 는 키:값 쌍 으로 값을 저장하는 자료 구조입니다 . 딕셔너리의 값은 모든 데이터 유형일 수 있으며 복제될 수 있지만, 키는 반복될 수 없으며 **변경 불가능** 해야 합니다 .

예: 여기에서는 데이터가 **키:값** 쌍으로 사전에 저장되어 있어 값을 찾기가 더 쉽습니다 .

```
d = {1: 'Geeks', 2: 'For', 3: 'Geeks'}
print(d)
```

사전을 만드는 방법

사전은 중괄호 {} 안에 일련의 요소를 넣고 '쉼표'로 구분하여 만들 수 있습니다.

```
d1 = {1: 'Geeks', 2: 'For', 3: 'Geeks'}
print(d1)

# create dictionary using dict() constructor
d2 = dict(a = "Geeks", b = "for", c = "Geeks")
print(d2)
```

- **사전 키는 대소문자를 구분합니다.** 즉, 키의 이름은 같지만 대소문자가 다르면 별도로 처리됩니다.
- **키는 변경 불가능해야 합니다.** 즉, 키는 문자열, 숫자 또는 튜플일 수 있지만 목록은 될 수 없습니다.
- **키는 고유해야 합니다.** 중복 키는 허용되지 않으며 중복 키는 이전 값을 덮어씁니다.

사전 항목 액세스

대괄호 안에 있는 **키를 사용하거나** g et() 메서드를 사용하여 사전에서 값에 접근할 수 있습니다 .

```
d = { "name": "Prajjwal", 1: "Python", (1, 2): [1,2,4] }

# Access using key
print(d["name"])

# Access using get()
print(d.get("name"))
```

사전 항목 추가 및 업데이트

할당을 사용하여 새로운 키-값 쌍을 추가하거나 기존 키를 업데이트할 수 있습니다.

```
d = {1: 'Geeks', 2: 'For', 3: 'Geeks'}

# Adding a new key-value pair
d["age"] = 22

# Updating an existing value
d[1] = "Python dict"

print(d)
```

사전 항목 제거

다음 방법을 사용하여 사전에서 항목을 제거할 수 있습니다.

- del : 키로 항목을 제거합니다.
- pop() : 키로 항목을 제거하고 해당 값을 반환합니다.
- clear() : 사전을 비웁니다.
- popitem() : 마지막 키-값 쌍을 제거하고 반환합니다.

```
d = {1: 'Geeks', 2: 'For', 3: 'Geeks', 'age':22}

# Using del to remove an item
del d["age"]
print(d)

# Using pop() to remove an item and return the value
val = d.pop(1)
print(val)

# Using popitem to removes and returns
# the last key-value pair.
key, val = d.popitem()
print(f"Key: {key}, Value: {val}")

# Clear all items from the dictionary
d.clear()
print(d)
```

사전 반복

for 루프를 사용하면 **키** (keys() 메서드 사용) **나 값** (values() 메서드 사용) 또는 둘 다(item() 메서드 사용) 반복할 수 있습니다 .

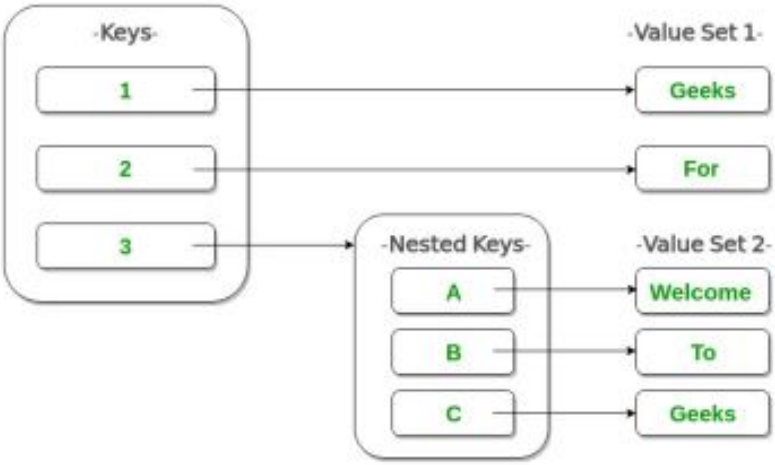
```
d = {1: 'Geeks', 2: 'For', 'age':22}

# Iterate over keys
for key in d:
    print(key)

# Iterate over values
for value in d.values():
    print(value)

# Iterate over key-value pairs
for key, value in d.items():
    print(f"{key}: {value}")
```

중첩된 사전



중첩된 사전의 예:

```
d = {1: 'Geeks', 2: 'For',
        3: {'A': 'Welcome', 'B': 'To', 'C': 'Geeks'}}

print(d)
```

파이썬에서 사전 복사하기

사전의 복사본은 **얕은 복사** 또는 **깊은 복사** 방법을 사용하여 생성할 수 있습니다. 이러한 방법은 사전 객체를 복제할 수 있지만, 중첩된 데이터의 경우에는 다르게 동작합니다. 두 방법 모두 자세히 살펴보겠습니다.

1. 얕은 카피

얕은 **복사는** 원본과 동일한 외부 값을 갖는 새 사전을 만듭니다. 하지만 사전에 중첩된 데이터(예: 목록이나 다른 사전)가 있는 경우, 두 사본 모두 해당 내부 데이터를 공유합니다. 따라서 중첩된 부분을 변경하면 다른 부분에도 영향을 미칩니다.

이는 Python의 copy 모듈의 **copy.copy() 메서드를 사용하여 생성됩니다.**

예:

이 코드는 얕은 복사에서 중첩된 데이터의 변경 사항이 중첩된 부분이 공유되기 때문에 원본과 복사본 모두에 영향을 미친다는 것을 보여줍니다.

```
import copy
original = {'name': 'Alice', 'marks': {'math': 90, 'science': 95}}

# Create a shallow copy
shallow = copy.copy(original)

# Modify nested value in the copy
shallow['marks']['math'] = 100

print("Original:", original)
print("Shallow Copy:", shallow)
```

산출

```
원본: {'name': 'Alice', 'marks': {'math': 100, 'science': 95}}
얕은 복사: {'name': 'Alice', 'marks': {'math': 100, 'science': 95}}
```

설명:

- **shallow = copy.copy(original):** 얕은 복사본을 생성하고, 중첩된 ' **marks** '는 공유된 상태로 유지됩니다.
- **shallow['marks']['math'] = 100:** 공유 중첩 사전에서 ' **math** '**를 업데이트합니다.**
- **print(original), print(shallow):** 둘 다 공유된 데이터로 인해 업데이트된 ' **math** ' **값을 표시합니다.**

2. 딥 카피

깊은 **복사는** 새로운 사전을 만들고, 모든 중첩된 데이터(목록이나 다른 사전 등)의 별도 사본도 만듭니다. 즉, 원본과 사본은 완전히 독립적이며, 중첩된 부분을 변경해도 다른 부분에 영향을 미치지 않습니다.

이는 Python의 copy 모듈의 **copy.deepcopy() 메서드를 사용하여 생성됩니다.**

예:

이 예에서는 심층 복사가 외부 데이터와 중첩된 데이터 모두의 완전히 독립적인 복사본을 생성하므로 복사본의 변경 사항이 원본에 영향을 미치지 않는다는 것을 보여줍니다.

```
import copy
original = {'name': 'Alice', 'marks': {'math': 90, 'science': 95}}

# Create a deep copy
deep = copy.deepcopy(original)

# Modify nested value in the deep copy
deep['marks']['math'] = 100

print("Original:", original)
print("Deep Copy:", deep)
```

산출

```
원본: {'name': 'Alice', 'marks': {'math': 90, 'science': 95}}
심층 복사: {'name': '앨리스', 'marks': {'math': 100, 'science': 95}}
```

설명:

- **deep = copy.deepcopy(original):** 깊은 복사를 생성하고, 중첩된 ' **marks** '도 별도로 복사합니다.
- **deep['marks']['math'] = 100:** 깊은 복사본의 중첩된 사전에서만 ' **math** '**를 업데이트합니다.**
- **print(original), print(deep):** original은 변경되지 않고, deep copy만이 업데이트된 ' **math** ' 값을 보여줍니다.